

Lab 7: Multiple Linear Regression

Author: Date: 2026-05-11

Topic Outline:

- Multiple Linear Regression:
 - Extending SLR to multiple predictors.
 - Fitting a model using `statsmodels`.
 - Interpreting model output: coefficients, adjusted R^2 , and the F-test.
 - Visualising the fit: actual vs predicted plots.
 - Making new predictions.
 - Residual analysis and model diagnostics.
-

Collaboration

You are encouraged to collaborate with other students in your labs, but you are expected to write up your own work for submission. Please do not copy and paste other people's solutions to problems as it is considered plagiarism and you will be penalized and reported. Should you choose to collaborate with others, please note their names here:

Your name:

Collaborators:

1.

Agent Usage

Additionally, you are permitted to use resources such as ChatGPT and Claude to help you with your lab assignments and to enhance your learning experience. Please make a note of any agents you have used in this submission here:

Agents:

1. Introduction

In Lab 6 we introduced **Simple Linear Regression (SLR)**, which models the relationship between a single predictor x and a response y . In practice, most phenomena are influenced by *several* variables simultaneously. **Multiple Linear Regression (MLR)** extends SLR by including p predictor variables $x_1, x_2, \dots, x_p$:

$$Y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \dots + \beta_p x_{ip} + \varepsilon_i, \quad \varepsilon_i \stackrel{\text{iid}}{\sim} N(0, \sigma^2).$$

The OLS Estimator

It is convenient to write the MLR model for all n observations simultaneously in **matrix form**:

$$\mathbf{Y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\varepsilon},$$

where

$$\mathbf{Y} = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix}, \quad \mathbf{X} = \begin{pmatrix} 1 & x_{11} & \cdots & x_{1p} \\ 1 & x_{21} & \cdots & x_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n1} & \cdots & x_{np} \end{pmatrix}, \quad \boldsymbol{\beta} = \begin{pmatrix} \beta_0 \\ \beta_1 \\ \vdots \\ \beta_p \end{pmatrix}, \quad \boldsymbol{\varepsilon} = \begin{pmatrix} \varepsilon_1 \\ \varepsilon_2 \\ \vdots \\ \varepsilon_n \end{pmatrix}$$

The matrix $\mathbf{X}$ is called the **design matrix** — each row is one observation and the leading column of ones accommodates the intercept β_0 .

The OLS estimator minimises the Residual Sum of Squares $\text{RSS} = \|\mathbf{Y} - \mathbf{X}\boldsymbol{\beta}\|^2$. Differentiating with respect to $\boldsymbol{\beta}$ and setting the result to zero yields the **normal equations**:

$$\mathbf{X}^\top \mathbf{X} \hat{\boldsymbol{\beta}} = \mathbf{X}^\top \mathbf{Y}.$$

Provided $\mathbf{X}^\top \mathbf{X}$ is invertible (i.e. no predictor is an exact linear combination of the others), the unique solution is:

$$\hat{\boldsymbol{\beta}} = \left(\mathbf{X}^\top \mathbf{X} \right)^{-1} \mathbf{X}^\top \mathbf{Y}.$$

This closed-form expression gives the OLS coefficient estimates for all $p + 1$ parameters simultaneously. The fitted values and residuals are then:

$$\hat{\mathbf{Y}} = \mathbf{X}\hat{\boldsymbol{\beta}}, \quad \mathbf{e} = \mathbf{Y} - \hat{\mathbf{Y}}.$$

Statistical Inference

As before, the coefficients are estimated by **Ordinary Least Squares (OLS)**,

minimising the Residual Sum of Squares:

$$\text{RSS} = \sum_{i=1}^n (y_i - \hat{y}_i)^2, \quad \hat{y}_i = \hat{\beta}_0 + \hat{\beta}_1 x_{i1} + \cdots + \hat{\beta}_p x_{ip}.$$

Adjusted R^2 . Adding predictors always increases the ordinary R^2 , even if those predictors are useless. The **adjusted R^2** penalises for model complexity:

$$\bar{R}^2 = 1 - \frac{\text{RSS}/(n - p - 1)}{\text{TSS}/(n - 1)}.$$

It can decrease when an additional predictor contributes less than expected by chance, making it a more honest measure of fit when comparing models with different numbers of predictors.

The F-test. The overall F-test asks whether *at least one* $\beta_j \neq 0$:

$$H_0 : \beta_1 = \beta_2 = \cdots = \beta_p = 0 \quad \text{vs} \quad H_1 : \text{at least one } \beta_j \neq 0.$$

A significant F-test means the model as a whole explains meaningful variation in Y , even if some individual predictors are not significant on their own.

In this lab we will:

1. Introduce two datasets — a worked penguin body-mass example and a California housing exercise.
2. Fit MLR models using `statsmodels` and interpret the output.
3. Visualise the fit using *actual vs predicted* plots.
4. Make predictions for new observations.
5. Perform residual analysis to assess model adequacy.

For this lab we will require the following packages:

```
In [2]: # Packages
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import statsmodels.formula.api as smf
from statsmodels.graphics.gofplots import qqplot
from sklearn.datasets import fetch_california_housing

# Seaborn style
sns.set_style("whitegrid")
sns.set_palette("Set2")
```

2. The Datasets

In this lab we consider two datasets. The first is the **Palmer Penguins** dataset, which we use as a worked example to illustrate the full MLR workflow. The second is the **California Housing** dataset from `scikit-learn`, which you will work through in the exercises.

2.1 Dataset 1: Palmer Penguins

We are familiar with the Palmer Penguins data set from previous labs and from lecture. Last lab we fit a SLR of `body_mass_g` against `flipper_length_mm`. Now we will look to incorporate multiple covariates. Below we load the data

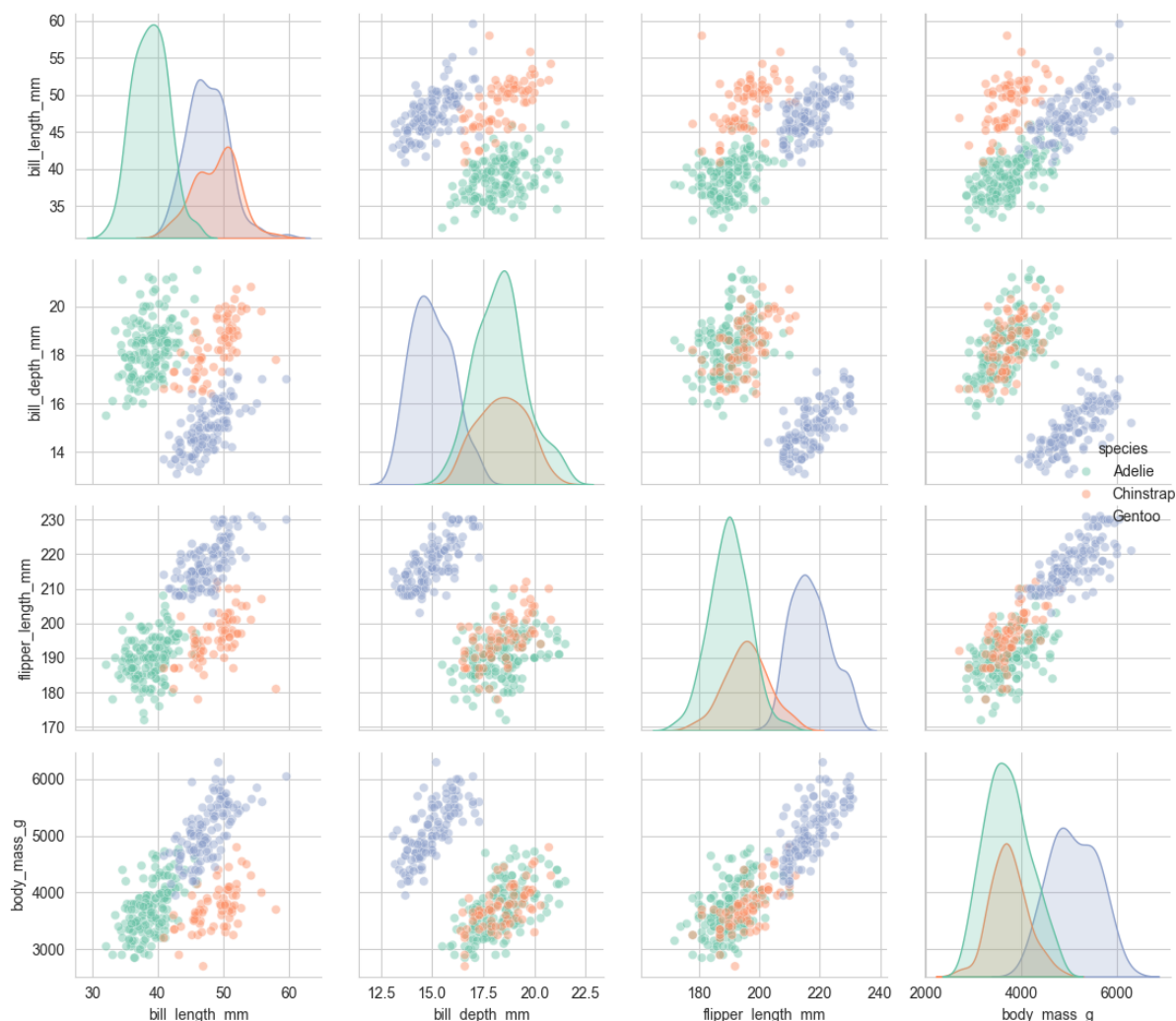
```
In [3]: # Load penguins, drop rows with missing values
df_penguins = sns.load_dataset('penguins').dropna()
df_penguins.describe().round(2)
```

```
Out[3]:
```

	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g
count	333.00	333.00	333.00	333.00
mean	43.99	17.16	200.97	4207.06
std	5.47	1.97	14.02	805.22
min	32.10	13.10	172.00	2700.00
25%	39.50	15.60	190.00	3550.00
50%	44.50	17.30	197.00	4050.00
75%	48.60	18.70	213.00	4775.00
max	59.60	21.50	231.00	6300.00

```
In [4]: # Pairplot to visualise all pairwise relationships
sns.pairplot(df_penguins, plot_kws={'alpha': 0.45}, hue='species')
plt.suptitle('Pairplot of Palmer Penguins Measurements', y=1.02)
plt.tight_layout()
plt.show()
```

Pairplot of Palmer Penguins Measurements



The pairplot reveals that `flipper_length_mm` has the strongest positive linear relationship with `body_mass_g`. Both `bill_length_mm` and `bill_depth_mm` also correlate with body mass, though less strongly. Notably, the three predictors are correlated with each other — particularly `bill_length_mm` and `flipper_length_mm` — a phenomenon called **multicollinearity** that we will revisit when interpreting the model summary.

2.2 Dataset 2: California Housing

The California Housing dataset (from `scikit-learn`) contains census data for 20,640 California districts, collected in 1990. The response variable is `MedHouseVal`, the **median house value in units of \$100,000**. The dataset contains the following variables:

Variable	Description
<code>MedInc</code>	Median household income in the district (tens of thousands of

dollars)

HouseAge	Median age of houses in the district (years)
AveRooms	Average number of rooms per household in the district
AveBedrms	Average number of bedrooms per household in the district
Population	Total population of the district
AveOccup	Average number of occupants per household in the district
Latitude	Latitude of the district centroid (degrees north)
Longitude	Longitude of the district centroid (degrees west, reported as negative)
MedHouseVal	Response: median house value in the district (units of \$100,000)

```
In [5]: # Load California Housing from scikit-learn
housing_data = fetch_california_housing()
df_housing = pd.DataFrame(housing_data.data, columns=housing_data.feature_names)
df_housing['MedHouseVal'] = housing_data.target
df_housing.describe().round(3)
```

```
Out[5]:
```

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup
count	20640.000	20640.000	20640.000	20640.000	20640.000	20640.000
mean	3.871	28.639	5.429	1.097	1425.477	3.071
std	1.900	12.586	2.474	0.474	1132.462	10.386
min	0.500	1.000	0.846	0.333	3.000	0.692
25%	2.563	18.000	4.441	1.006	787.000	2.430
50%	3.535	29.000	5.229	1.049	1166.000	2.818
75%	4.743	37.000	6.052	1.100	1725.000	3.282
max	15.000	52.000	141.909	34.067	35682.000	1243.333

MedHouseVal ranges from 0.15 to 5.00 (i.e. 15,000 to 500,000), with a mean of about \$206,900. **MedInc** has a wide spread (0.5 to 15.0), while **AveRooms** and **AveOccup** have extreme upper outliers that likely correspond to unusual districts (e.g. hotels or institutions). Keep these features of the data in mind when interpreting model output.

!! Problem

Q0 Exploring the Dataset

1. Display the full dataset description by printing `housing_data.DESCR`.
Read through it and note what you learn about how the data were collected.
2. Produce a figure with **three side-by-side histograms** showing the distributions of `AveRooms`, `AveBedrms`, and `AveOccup`. Label each axis clearly and give the figure an overall title. Comment briefly on the shape of each distribution — in particular, note any extreme outliers and what they might represent.

Solution:

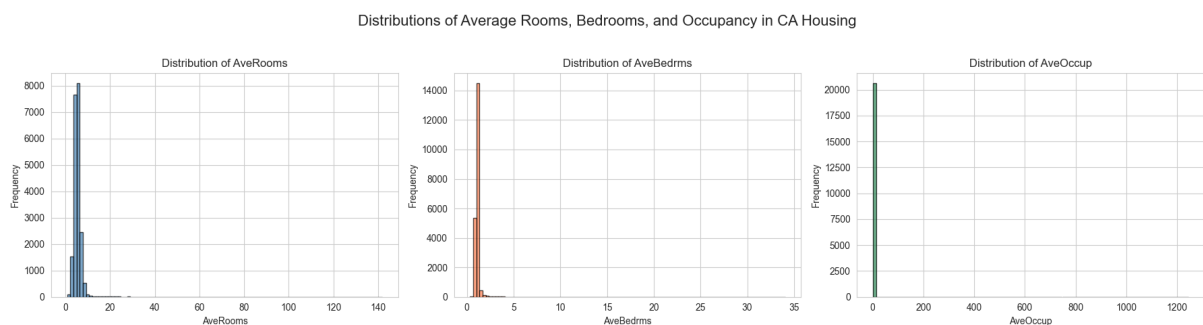
```
In [ ]: print(housing_data.DESCR)
# The dataset contains 20,640 observations, each representing a unique
# the median house value was capped at 5.0, $500,000, this means any b
# there are large outliers for the number of rooms, as you can see wit
```

```
In [6]: fig, axes = plt.subplots(1, 3, figsize=(18, 5))

# List of features to plot
features = ['AveRooms', 'AveBedrms', 'AveOccup']
colors = ['steelblue', 'coral', 'seagreen']

for i, feature in enumerate(features):
    axes[i].hist(df_housing[feature], bins=100, color=colors[i], edgecolor='black')
    axes[i].set_title(f'Distribution of {feature}')
    axes[i].set_xlabel(feature)
    axes[i].set_ylabel('Frequency')

plt.suptitle('Distributions of Average Rooms, Bedrooms, and Occupancy')
plt.tight_layout(rect=[0, 0.03, 1, 0.95])
plt.show()
```



These graphs all show extreme outliers which make it difficult to observe the data as the max values are extreme compared to all other values

3. Fitting Multiple Linear Regression Models

In practice we do not invert $\mathbf{X}^\top \mathbf{X}$ by hand — `statsmodels` handles this for us. Extending the formula API from SLR to MLR is straightforward: simply join the predictor names with `+`:

```
model = smf.ols('response ~ predictor1 + predictor2 +
predictor3', data=df).fit()
statsmodels automatically constructs the design matrix (including
```

Recall from lecture that we found the incorporation of dummy variables for species to improve our model fit. We therefore fit:

$$\text{body_mass_g}_i = \beta_0 + \beta_1 \cdot \text{bill_length}_i + \beta_2 \cdot \text{bill_depth}_i + \beta_3 \cdot \text{flipper_length}_i$$

where $\mathbf{1}[\text{Chinstrap}]_i$ and $\mathbf{1}[\text{Gentoo}]_i$ are **dummy variables** that equal 1 when penguin i belongs to the Chinstrap or Gentoo species respectively, and 0 otherwise. **Adelie is the reference category** — its baseline body mass is captured by the intercept β_0 . The coefficients β_4 and β_5 each measure the expected difference in body mass between that species and an Adelie, **holding bill length, bill depth, and flipper length fixed**. `statsmodels` selects the reference category alphabetically and labels the dummy columns `C(species)` `[T.Chinstrap]` and `C(species)[T.Gentoo]` in the summary output.

and inspect the output:

```
In [7]: # Fit MLR model for penguins (species included as dummy variables)
model_p = smf.ols(
    'body_mass_g ~ bill_length_mm + bill_depth_mm + flipper_length_mm
    data=df_penguins
).fit()
print(model_p.summary())
```

OLS Regression Results

```
=====
=====
Dep. Variable:          body_mass_g    R-squared:
0.849
Model:                  OLS           Adj. R-squared:
0.847
Method:                 Least Squares   F-statistic:
369.1
Date:                   Mon, 11 May 2026   Prob (F-statistic):
22e-132
Time:                   17:30:58          Log-Likelihood:
-2384.8
No. Observations:      333              AIC:
4782.
Df Residuals:          327              BIC:
4805.
```

```

Df Model:                    5
Covariance Type:            nonrobust
=====
=====
                                coef    std err          t      P>|t|
[0.025    0.975]
-----
Intercept                    -4282.0802    497.832     -8.601    0.000
5261.438   -3302.723
C(species)[T.Chinstrap]    -496.7583     82.469     -6.024    0.000
-658.995   -334.521
C(species)[T.Gentoo]       965.1983    141.770      6.808    0.000
686.301    1244.096
bill_length_mm              39.7184      7.227      5.496    0.000
25.501     53.936
bill_depth_mm              141.7714     19.163      7.398    0.000
104.072    179.470
flipper_length_mm          20.2264      3.135      6.452    0.000
14.059     26.394
=====
=====
Omnibus:                     7.321    Durbin-Watson:
2.247
Prob(Omnibus):               0.026    Jarque-Bera (JB):
7.159
Skew:                        0.348    Prob(JB):
0.0279
Kurtosis:                    3.179    Cond. No.
6.01e+03
=====
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 6.01e+03. This might indicate that there are strong multicollinearity or other numerical problems.

The key quantities to extract from the summary are:

Field	Location in output	Interpretation
coef — Intercept	Coefficients table	$\hat{\beta}_0$: predicted body mass for an Adelie when all continuous predictors equal zero (not physically meaningful)
coef — continuous predictors	Coefficients table	$\hat{\beta}_j$: expected change in body mass (g) per 1-unit increase in x_j , holding species and all other predictors fixed

<code>coef — C(species) [T.Chinstrap]</code>	Coefficients table	$\hat{\beta}_4$: expected difference in body mass between a Chinstrap and an Adelie with the same bill and flipper measurements
<code>coef — C(species) [T.Gentoo]</code>	Coefficients table	$\hat{\beta}_5$: expected difference in body mass between a Gentoo and an Adelie with the same bill and flipper measurements
<code>P>\ t\ </code>	Coefficients table	p-value for $H_0 : \beta_j = 0$
Adj. R-squared	Top block	R^2 penalised for number of predictors — more honest for MLR
F-statistic	Top block	Test that at least one $\beta_j \neq 0$

Interpreting this model:

- **Species dummies:** the large positive coefficient on `C(species) [T.Gentoo]` reflects the well-known fact that Gentoos are substantially heavier than Adelies for a given set of bill and flipper measurements. The Chinstrap coefficient is much smaller in magnitude, indicating Chinstraps are similar in mass to Adelies once bill and flipper size are controlled for. Both species effects are highly significant.
- **Continuous predictors:** notice that once species is included, the coefficients on `bill_length_mm` and `bill_depth_mm` change substantially compared to the species-free model — and become significant. This is because species was acting as a **confounder**: without it in the model, the bill measurements were partly proxying for between-species differences in body size rather than within-species variation.
- **Adjusted R^2 :** the model explains considerably more variance in body mass than the species-free version (which had $\bar{R}^2 = 0.762$), confirming what was demonstrated in lecture — species is an important predictor that should not be omitted.
- **The F-test** ($p \approx 0$) confirms the model as a whole is highly significant.

!! Problem

Q1 Fitting a Model

Fit an MLR model to the California Housing data with `MedHouseVal` as the response and `MedInc`, `HouseAge`, `AveRooms`, and `AveOccup` as predictors.

Print the model summary and answer the following:

1. Which predictor has the largest (in magnitude) t-statistic? What does this tell us?
 2. Interpret the coefficient on `AveRooms`. Does its sign surprise you? Why might it be negative when holding the other variables fixed?
 3. Report the adjusted R^2 . What percentage of the variance in median house value is explained by these four predictors?
 4. What does the F-test tell us?
-

Solution:

```
In [8]: model_housing = smf.ols(  
        formula = 'MedHouseVal ~ MedInc + HouseAge + AveRooms + AveOccup',  
        data = df_housing  
    ).fit()  
model_housing.summary()
```

Out [8]:

OLS Regression Results

Dep. Variable:	MedHouseVal	R-squared:	0.514
Model:	OLS	Adj. R-squared:	0.514
Method:	Least Squares	F-statistic:	5450.
Date:	Tue, 12 May 2026	Prob (F-statistic):	0.00
Time:	21:15:13	Log-Likelihood:	-24802.
No. Observations:	20640	AIC:	4.961e+04
Df Residuals:	20635	BIC:	4.965e+04
Df Model:	4		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
Intercept	0.0314	0.022	1.428	0.153	-0.012	0.074
MedInc	0.4433	0.003	141.672	0.000	0.437	0.449
HouseAge	0.0169	0.000	37.359	0.000	0.016	0.018
AveRooms	-0.0273	0.002	-11.330	0.000	-0.032	-0.023
AveOccup	-0.0045	0.001	-8.267	0.000	-0.006	-0.003

Omnibus:	4419.990	Durbin-Watson:	0.807
Prob(Omnibus):	0.000	Jarque-Bera (JB):	11655.492
Skew:	1.158	Prob(JB):	0.00
Kurtosis:	5.862	Cond. No.	126.

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

1. Median income with a t value of 141.672 which tells me it is a statistically significant predictor of house values
2. The coefficient for AveRooms is -0.0273. For every additional room added to the average house in a block group, the median house value is predicted to decrease by approximately \$2,730 when holding the other values fixed. It may be negative because homes with more rooms don't necessarily mean

they are more expensive

3. The Adjusted R^2 is 0.514. 51.4% of the variance in California median house values is explained by these four predictors. This is a good predictive power but is still unexplained.
4. Because the p-value is 0.00, we reject that null hypothesis. This confirms that your model is statistically significant and that all variables in this case help explain house values better than a model with no predictors at all.

4. Visualising the Fit: Actual vs Predicted

With a single predictor we could plot the fitted line directly on a scatter plot. With multiple predictors this is no longer possible — the fitted surface lives in a higher-dimensional space. Instead, the standard visualisation for MLR is the **actual vs predicted plot**: a scatter plot of the observed response y_i against the fitted values $\hat{y}_i$.

If the model fits well, the points should cluster tightly around the **diagonal reference line** $y = \hat{y}$ (perfect prediction). Systematic deviations — such as curvature, fan shapes, or points concentrated above or below the line — signal model misspecification or heteroscedasticity.

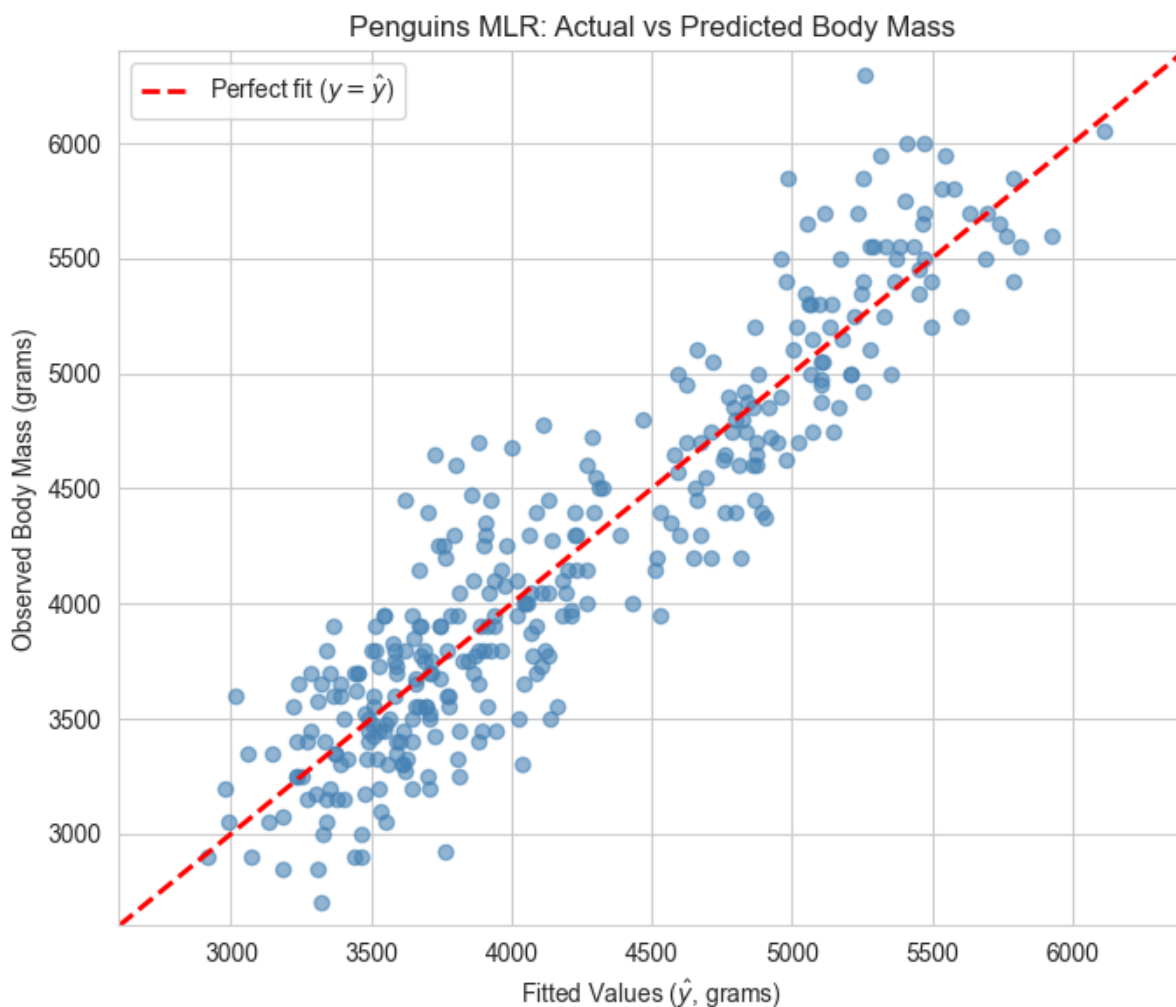
The fitted values are stored in `model.fittedvalues` after calling `.fit()`.

For the penguins model we plot observed body mass against fitted body mass:

```
In [9]: # Actual vs Predicted for penguins
fig, ax = plt.subplots(figsize=(7, 6))
ax.scatter(model_p.fittedvalues, df_penguins['body_mass_g'], alpha=0.6)

# Diagonal reference line
lims = [
    min(model_p.fittedvalues.min(), df_penguins['body_mass_g'].min()),
    max(model_p.fittedvalues.max(), df_penguins['body_mass_g'].max())
]
ax.plot(lims, lims, 'r--', lw=2, label='Perfect fit ($y = \hat{y}$)')
ax.set_xlim(lims)
ax.set_ylim(lims)
ax.set_xlabel('Fitted Values ($\hat{y}$, grams)')
ax.set_ylabel('Observed Body Mass (grams)')
ax.set_title('Penguins MLR: Actual vs Predicted Body Mass')
ax.legend()
plt.tight_layout()
```

```
plt.show()
```



The points cluster more tightly around the diagonal than they did in the species-free model, consistent with the higher adjusted R^2 achieved by including species. The spread is roughly constant across the range of fitted values with no obvious curvature — a positive sign for the linearity and homoscedasticity assumptions. Any remaining tendency to under-predict the heaviest or over-predict the lightest penguins will be investigated in the residual analysis.

!! Problem

Q2 Actual vs Predicted Plot

Produce an actual vs predicted plot for the California Housing model. Include the diagonal reference line. Comment on:

1. How tightly the points cluster around the diagonal (does this match the R^2 value?).
2. Whether there is any systematic pattern (curvature, fan shape, clustering

above or below the line).

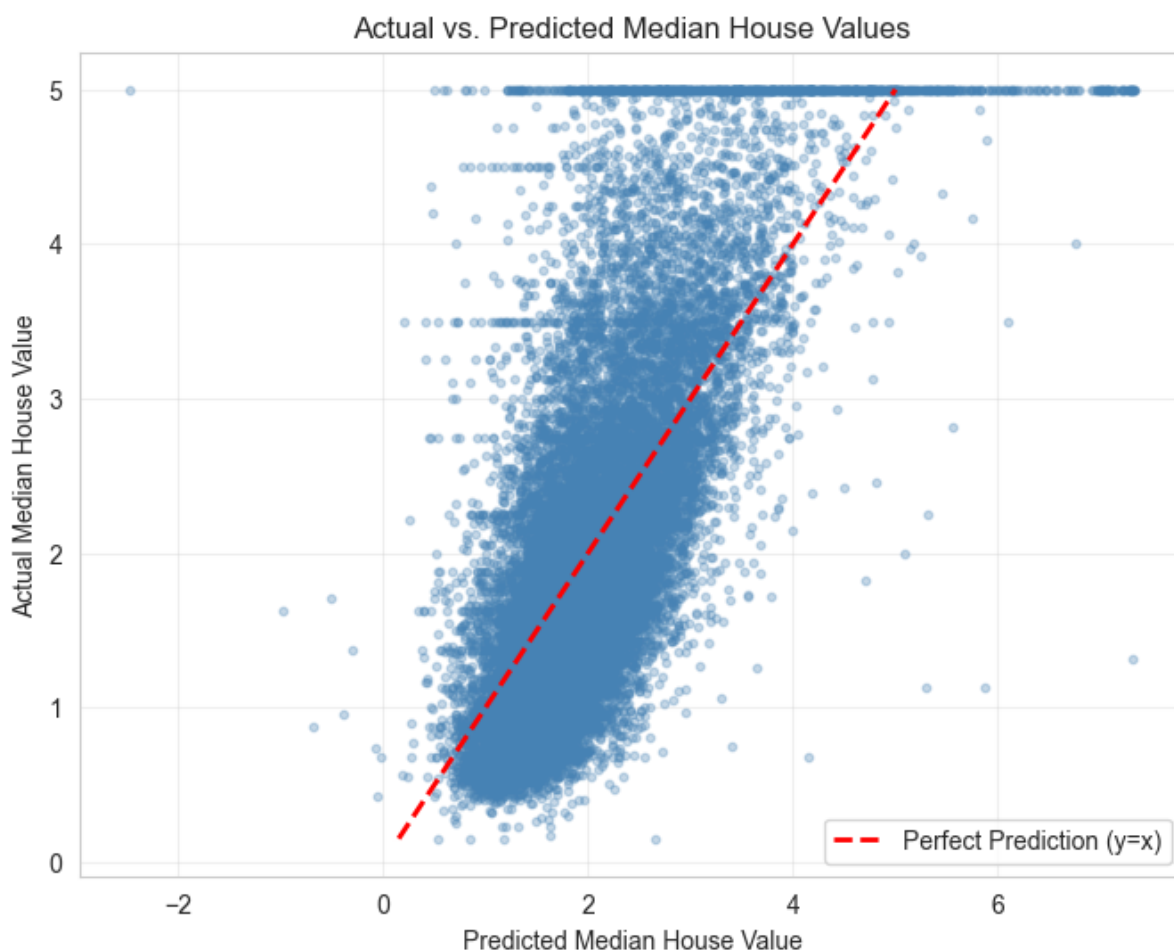
Solution:

```
In [14]: y_pred = model_housing.fittedvalues
y_actual = df_housing['MedHouseVal']

plt.figure(figsize=(8, 6))
plt.scatter(y_pred, y_actual, alpha=0.3, color='steelblue', s=10)

line_coords = np.linspace(min(y_actual), max(y_actual), 100)
plt.plot(line_coords, line_coords, color='red', linestyle='--', lw=2,

plt.xlabel('Predicted Median House Value')
plt.ylabel('Actual Median House Value')
plt.title('Actual vs. Predicted Median House Values')
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```



The points follow the line but not perfectly which I feel like is a good representation of the R^2 value being .5. There is a weird pattern going on at the 5 line on the graph where it looks like a straight line. Also the data is scattered

unevenly at the top compared to the lower half of the graph.

5. Making New Predictions

Predictions in MLR work exactly as in SLR: pass a `DataFrame` containing the new predictor values (with the same column names as in the model formula) to `.get_prediction().summary_frame()`. The key difference is that we now must specify a value for **each** predictor in the model.

```
In [15]: # New penguins to predict body mass for (species must be specified)
new_penguins = pd.DataFrame({
    'bill_length_mm': [39.5, 46.5, 52.0],
    'bill_depth_mm': [15.7, 17.9, 19.5],
    'flipper_length_mm': [187, 203, 221],
    'species': ['Adelie', 'Chinstrap', 'Gentoo']
})

# Full prediction summary: point estimate + CI + PI
pred_p = model_p.get_prediction(new_penguins).summary_frame(alpha=0.05)
pred_p.index = ['Penguin A (Adelie)', 'Penguin B (Chinstrap)', 'Penguin C (Gentoo)']

# Select and rename for readability
pred_p = pred_p[['mean', 'mean_ci_lower', 'mean_ci_upper', 'obs_ci_low', 'obs_ci_high']]
pred_p.columns = ['Body Mass (g)', 'CI Lower', 'CI Upper', 'PI Lower', 'PI Upper']
pred_p.round(1)
```

```
Out[15]:
```

	Body Mass (g)	CI Lower	CI Upper	PI Lower	PI Upper
Penguin A (Adelie)	3295.0	3182.3	3407.6	2665.6	3924.3
Penguin B (Chinstrap)	3711.7	3611.6	3811.9	3084.5	4339.0
Penguin C (Gentoo)	5983.1	5824.9	6141.2	5344.0	6622.1

The three hypothetical penguins each belong to a different species, which now contributes directly to the prediction alongside the continuous measurements. Note that when making predictions from a model that includes species, a species value must be specified for each new observation — the model cannot impute it.

- **Penguin A (Adelie)** has small bills and short flippers typical of an Adelie; the species dummy contributes zero (Adelie is the reference category), so the prediction is driven entirely by the continuous terms.
- **Penguin B (Chinstrap)** has mid-range measurements; the Chinstrap dummy

shifts the prediction by $\hat{\beta}_4$ relative to an Adelie with the same measurements.

- **Penguin C (Gentoo)** has large bills and long flippers typical of a Gentoo; the large positive Gentoo dummy pushes the predicted mass substantially higher than the continuous measurements alone would suggest.

As before, prediction intervals are considerably wider than confidence intervals, and all predicted values should fall within the observed range of `body_mass_g` (2,700 g – 6,300 g), confirming we are interpolating rather than extrapolating.

!! Problem

Q3 Making Predictions

Use your fitted housing model to predict the median house value (and associated 95% CI and PI) for the four districts described below. Present the results in a readable table and comment on:

1. How does predicted house value change as `MedInc` increases from 2.5 to 12.0?
2. Home A's prediction interval extends below zero. What does this mean, and why does it happen?

District	MedInc	HouseAge	AveRooms	AveOccup
Home A	2.5	10	4.0	2.5
Home B	5.0	25	5.5	2.8
Home C	8.0	35	6.5	3.0
Home D	12.0	50	7.0	3.5

Solution:

```
In [ ]: new_districts = pd.DataFrame({
    'MedInc': [2.5, 5.0, 8.0, 12.0],
    'HouseAge': [10, 25, 35, 50],
    'AveRooms': [4.0, 5.5, 6.5, 7.0],
    'AveOccup': [2.5, 2.8, 3.0, 3.5]
})

pred_h = model_housing.get_prediction(new_districts).summary_frame(alpha=.95)
pred_h.index = ['Home A', 'Home B', 'Home C', 'Home D']
pred_h = pred_h[['mean', 'mean_ci_lower', 'mean_ci_upper', 'obs_ci_low', 'obs_ci_high']]
pred_h.columns = ['MedHouseVal', 'CI Lower', 'CI Upper', 'PI Lower', 'PI Upper']
```

```
pred_h.round(3)
```

Out[]:

	MedHouseVal	CI Lower	CI Upper	PI Lower	PI Upper
Home A	1.188	1.165	1.211	-0.39	2.766
Home B	2.507	2.494	2.520	0.93	4.085
Home C	3.978	3.950	4.005	2.40	5.555
Home D	5.988	5.934	6.043	4.41	7.567

1. The MedInc value changing from 2.5 to 12 makes the predicted house value increase substantially
2. The interval for home A extends below zero because the normal distribution doesn't know that there is a boundary at zero and this is what is predicted based off of normal distributions average deviation from the mean. It doesn't mean that a house can have a negative value, but rather that to model uncertainty it crosses the zero axis because of the necessary width of the confidence interval.

6. Residual Analysis

The assumptions of MLR are the same as for SLR:

1. **Linearity:** the true relationship between each x_j and y is linear (after conditioning on the other predictors).
2. **Independence:** the errors ε_i are independent.
3. **Homoscedasticity:** the errors have constant variance σ^2 .
4. **Normality:** the errors follow a normal distribution.

We diagnose these assumptions using the same two plots as in Lab 6:

- **Residuals vs Fitted plot:** residuals $e_i = y_i - \hat{y}_i$ plotted against fitted values $\hat{y}_i$. A random scatter around zero is ideal; systematic patterns indicate violated assumptions.
- **Normal Q-Q plot:** quantiles of the residuals against theoretical normal quantiles. Points close to the diagonal support the normality assumption.

We produce these diagnostic plots for the penguins model:

```
In [ ]: fig, axes = plt.subplots(1, 2, figsize=(12, 5))
```

```
# Residuals vs Fitted
axes[0].scatter(model_p.fittedvalues, model_p.resid, alpha=0.65, color
axes[0].axhline(0, color='red', linestyle='--', lw=1.5)
axes[0].set_xlabel('Fitted Values ( $\hat{y}$ )')
axes[0].set_ylabel('Residuals ( $e_i$ )')
axes[0].set_title('Residuals vs Fitted – Penguins')

# Normal Q-Q plot
qqplot(model_p.resid, line='s', ax=axes[1], alpha=0.65)
axes[1].set_title('Normal Q-Q Plot – Penguins')

plt.tight_layout()
plt.show()
```

For the penguins model with species:

- **Residuals vs Fitted:** the residuals scatter approximately randomly around zero with no strong curvature or fan shape — consistent with linearity and homoscedasticity. Including species removes the systematic between-species variation in body mass that would otherwise appear as structured clustering in the residuals.
- **Q-Q plot:** the points follow the diagonal well across the central bulk of the data. The slight deviations at the tails are mild and expected with a sample of this size.

Overall, the assumptions are adequately satisfied for this model.

!! Problem

Q4 Residual Analysis

Reproduce these diagnostic plots for the California Housing model. What patterns do you observe in each plot? What assumptions appear to be violated, and what does this suggest about the model?

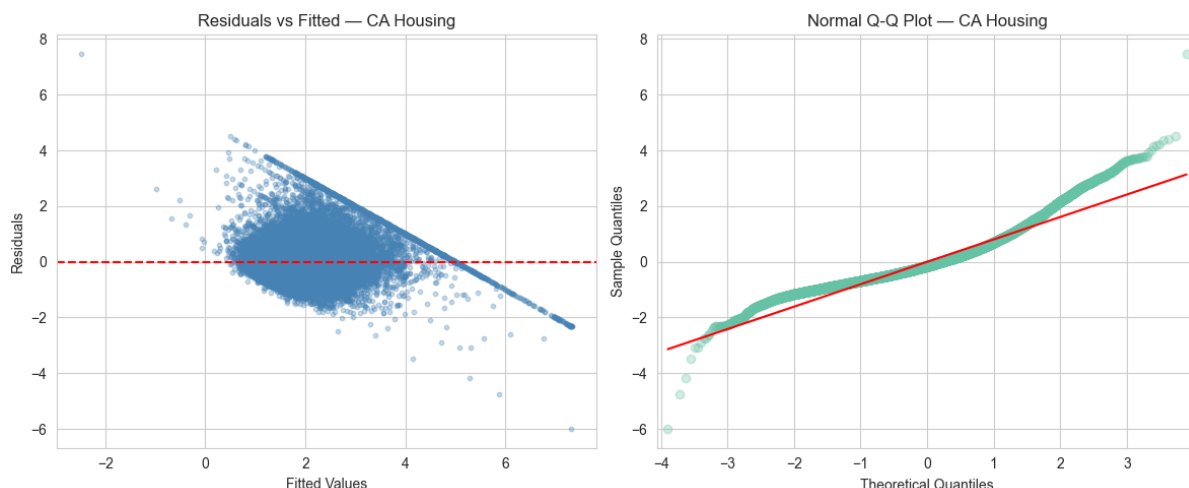
Solution:

```
In [19]: fig, axes = plt.subplots(1, 2, figsize=(12, 5))

# 1. Residuals vs Fitted
axes[0].scatter(model_housing.fittedvalues, model_housing.resid, alpha
axes[0].axhline(0, color='red', linestyle='--', lw=1.5)
axes[0].set_xlabel('Fitted Values')
axes[0].set_ylabel('Residuals')
axes[0].set_title('Residuals vs Fitted – CA Housing')
```

```
# 2. Normal Q-Q plot
qqplot(model_housing.resid, line='s', ax=axes[1], alpha=0.3)
axes[1].set_title('Normal Q-Q Plot - CA Housing')

plt.tight_layout()
plt.show()
```



- **Residuals vs Fitted:** the residuals scatter is not very consistent with linearity and homoscedasticity. There is a large cluster in the residuals and they have odd outliers. The diagonal pattern shows what I mentioned earlier about the 5.0 cap on values.
- **Q-Q plot:** the points follow the diagonal line possibly well across the data even though there is kinda a snake shape. I think there may be a violation of the normality assumption.

Overall, the model violated homoscedacity and linearity, and the model is maybe normal however it is difficult to examine from the graph

Submission Checklist

- Ensure you have completed all problems in the notebook.
- Clear your document code outputs and rerun the notebook from top to bottom, checking for any new errors.
- At the top of your notebook click more options -> export -> pdf.
 - Alternatively, export to html and save as a pdf.
- Make sure your pdf has all code chunks clearly visible and executed.
- Submit before the posted deadline.